

1.2 Source Code

```
#include <stdio.h>
#include <stdint.h>

// Simulated GPIO register
uint8_t GPIO_PORT = 0x00;

// GPIO pin number
#define LED_PIN 2

// Set a GPIO bit
void GPIO_SET(uint8_t pin)
{
    GPIO_PORT |= (1 << pin);
}

// Clear a GPIO bit
void GPIO_CLEAR(uint8_t pin)
{
    GPIO_PORT &= ~(1 << pin);
}

// Read a GPIO bit
int GPIO_READ(uint8_t pin)
{
    return (GPIO_PORT >> pin) & 1;
}

int main()
{
    printf("== GPIO BIT MANIPULATION ==\n\n");
    printf("Initial GPIO Register: 0x%02X\n", GPIO_PORT);

    // Set GPIO 2
    GPIO_SET(LED_PIN);
    printf("\nSetting GPIO 2...\n");
    printf("GPIO Register: 0x%02X\n", GPIO_PORT);
    printf("LED State: ON\n");

    // Read GPIO 2
    printf("GPIO 2 Value: %d\n", GPIO_READ(LED_PIN));

    // Clear GPIO 2
    GPIO_CLEAR(LED_PIN);
    printf("\nClearing GPIO 2...\n");
    printf("GPIO Register: 0x%02X\n", GPIO_PORT);
    printf("LED State: OFF\n");

    // Read GPIO 2
    printf("GPIO 2 Value: %d\n", GPIO_READ(LED_PIN));

    return 0;
}
```

1.3 Program Output

The program was compiled and executed successfully on OnlineGDB (C compiler). The captured console output is shown below.

```
== GPIO BIT MANIPULATION ==

Initial GPIO Register: 0x00

Setting GPIO 2...
GPIO Register: 0x04
LED State: ON
GPIO 2 Value: 1

Clearing GPIO 2...
GPIO Register: 0x00
LED State: OFF
GPIO 2 Value: 0

...Program finished with exit code 0
```

2.2 Source Code

```
#include <stdio.h>
#include <unistd.h>
#include <stdint.h>

volatile uint32_t tick = 0;

volatile int task1_flag = 0;
volatile int task2_flag = 0;
volatile int task3_flag = 0;

// Timer Interrupt Service Routine
void timer_ISR()
{
    tick++;

    // Task 1: every 500 ms
    if (tick % 5 == 0)
        task1_flag = 1;

    // Task 2: every 1 second
    if (tick % 10 == 0)
        task2_flag = 1;

    // Task 3: every 2 seconds
    if (tick % 20 == 0)
        task3_flag = 1;
}

// Task 1
void task1()
{
    printf("Task 1: LED toggled\n");
}

// Task 2
void task2()
{
    printf("Task 2: Sensor checked\n");
}

// Task 3
void task3()
{
    printf("Task 3: System status checked\n");
}

int main()
{
    printf("=== REAL-TIME TASK SCHEDULER ===\n\n");

    for (int i = 0; i < 40; i++)
    {
        // Generate timer interrupt
        timer_ISR();

        // Execute Task 1
        if (task1_flag)
        {
            task1_flag = 0;
            task1();
        }

        // Execute Task 2
        if (task2_flag)
        {
            task2_flag = 0;
            task2();
        }

        // Execute Task 3
        if (task3_flag)
        {
            task3_flag = 0;
            task3();
        }

        // Timer period = 100 ms
        usleep(100000);
    }

    printf("\nScheduler completed successfully.\n");
    return 0;
}
```

2.3 Program Output

The program was compiled and run on OnlineGDB. The complete console output for all 40 simulated timer ticks is shown below, exactly as produced by the run.

```
=== REAL-TIME TASK SCHEDULER ===

Task 1: LED toggled
Task 1: LED toggled
Task 2: Sensor checked
Task 1: LED toggled
Task 1: LED toggled
Task 2: Sensor checked
Task 3: System status checked
Task 1: LED toggled
Task 1: LED toggled
Task 2: Sensor checked
Task 1: LED toggled
Task 1: LED toggled
```

